ThermoGyro: Learning and Removing Temperature Drift in a Low-Cost MEMS Gyroscope

Course project · Introduction to AI & Machine Learning for Applications in Mechanical Engineering · NIT Tiruchirappalli

@Vaibhav · Oct 29, 2026

Summary

A cheap motion sensor gives wrong readings when it heats up, and machine learning can correct most of that error. In one real test, a sensor that was not moving at all "thought" it had turned 1,633° in 30 minutes as it warmed from 31 °C to 68 °C. With the correction built in this project, called **ThermoGyro**, the error dropped to 41°.

The sensor. The MPU-6050 is the small motion sensor used in my Best Project bicycle, in most college robots and in many drones. Its gyroscope measures how fast it is turning. When it is perfectly still it should read zero, but it doesn't: it reads a small error called the **bias**. The bias changes with the chip's temperature.

The data. I used real recordings, published by Cranfield University, of three MPU-6050 chips kept still inside a temperature chamber from −18 °C to 72 °C. That is 189,402 readings, all freely available.

What I built. I wrote every model myself in Python, using only what we learned in the course:

- a simple "measure once and subtract" method (what most hobby code does)
- straight-line and curved (polynomial) fits, solved with the normal equation
- regularisation, to stop curves from going wild
- gradient descent
- a small neural network, trained with backpropagation
- **ThermoGyro**, a hybrid: a simple curve plus a neural network that fixes what the curve misses

What I found.

- Within the temperatures it was calibrated on, ThermoGyro was the most accurate: 1.75 times better than the simple curve and 14 times better than "measure once".

- At temperatures it had never seen, the neural network on its own failed badly, and very wiggly curves gave crazy answers (up to 16,792 °/s wrong). The simple curve stayed reliable. So ThermoGyro switches its neural part off when it is outside the temperatures it knows.
- Every time the sensor is switched on, its error jumps a little (up to 2.7 °/s). So the bike measures its error for a few seconds at every start, and the model only handles the change after that.
- Every chip is different. A model from one chip does not work on another, so each sensor needs its own calibration.

What you can try. The project comes with a GitHub repository, a web app that runs the trained models on the real recordings and lets anyone calibrate their own sensor, and code for the ESP32 board.

1. The problem: one bike, a cool morning and a hot afternoon

The same bicycle sees very different temperatures in one day, and its sensor gets confused by that. In my first year I led the electronics of a Smart Dockless Bicycle Sharing System (ESP32, SIM800L GSM, NEO-6M GPS and an MPU-6050), which won Best Project at the NIT Trichy Energy Expo 2026. The MPU-6050 tells the bike which way it is facing and whether it has been moved or knocked over.

Think about one of those bikes on a normal day:

- **6:30 am, cool morning.** A student unlocks the bike. Like almost every Arduino program, the code measures the gyro's error once, while the bike stands still, and subtracts it from every reading after that.
- **1:30 pm, hot afternoon.** The bike has been standing in the sun. The sealed box with the electronics is now much hotter than the air, and the gyro's error is no longer the morning value.
- **What goes wrong.** To know which way it faces, the bike keeps adding up the gyro's readings every second. A small leftover error, added up again and again, makes a bike that is standing still look as if it is slowly turning. A line-follower robot, a balancing robot or a drone with the same chip has the same problem.

So the question for this project is: **can a model learn how this chip's error changes with temperature, so we can remove it at any temperature, cool or hot?**

Three more practical questions follow from it:

1. What if the bike is calibrated in the cool morning and used in the hot afternoon?
2. Does a model made one day still work the next day?
3. Can one chip's model be used on another chip?

Everything in this report comes from real recordings of real MPU-6050 chips. Nothing is simulated.

2. Why the gyro drifts when it gets hot

The gyro's zero point moves with temperature because both its tiny moving parts and its electronics change as the chip warms up. Inside the MPU-6050 is a tiny piece of silicon that is kept vibrating. When the chip turns, the vibration is pushed sideways (the Coriolis effect), and the chip measures that push as turning speed. Heat makes the silicon slightly softer, changes how it vibrates, puts stress on it where it is glued into its plastic case, and changes the electronics that read it. Each of these shifts the reading the chip gives when it is not turning at all.

The chip's datasheet gives the numbers:

What	Value from the MPU-6050 datasheet
How finely it measures (±250 °/s range)	131 steps per °/s
How much the zero error can change from −40 to +85 °C	up to ±20 °/s
Built-in temperature sensor	temperature (°C) = raw reading ÷ 340 + 36.53

The datasheet allows the error to change by up to 20 °/s, but it gives no formula for how. That is why the chip has its own thermometer: so that software can correct the error. In machine-learning terms, the job is to learn a function: **error = h(temperature)**.

Why a small error becomes a big problem. The bike works out its direction by adding up its turning speed over time. If the leftover error is only 1 °/s, then after 30 minutes the bike believes it has turned 1 × 60 × 30 = 1,800°, which is five full circles, while it has not moved at all. A good model of the error keeps that leftover close to zero at every temperature.

This is exactly the kind of problem Module 1 describes for data-driven modelling: we know roughly why the error happens, but no formula can predict it for a particular chip, so we must learn it from measurements. The physics still helps, because it tells us the curve should be smooth and gently bent, not wiggly.

3. The data we used

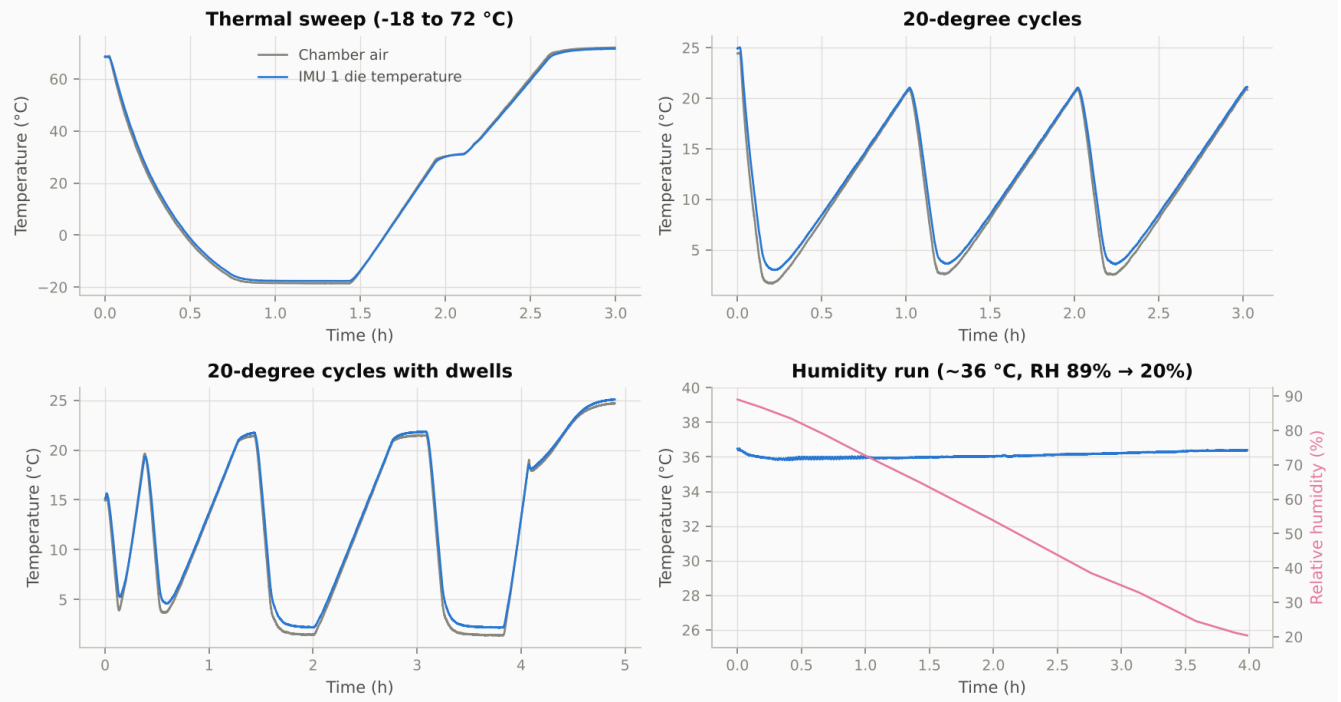
All the data is real: three MPU-6050 chips recorded by Cranfield University while kept completely still inside a temperature chamber. The dataset is Low-cost IMU Sensor Temperature Humidity Zero Bias Data by Maton, Economou, Galvão Wall, Khan and Cooper (2024). It is free to use with credit (licence CC BY 4.0).

Because the chips never turned, everything the gyro reports is error plus random noise. That is exactly what we want to model. Each reading also includes the chip's own temperature, which is the input our models use.

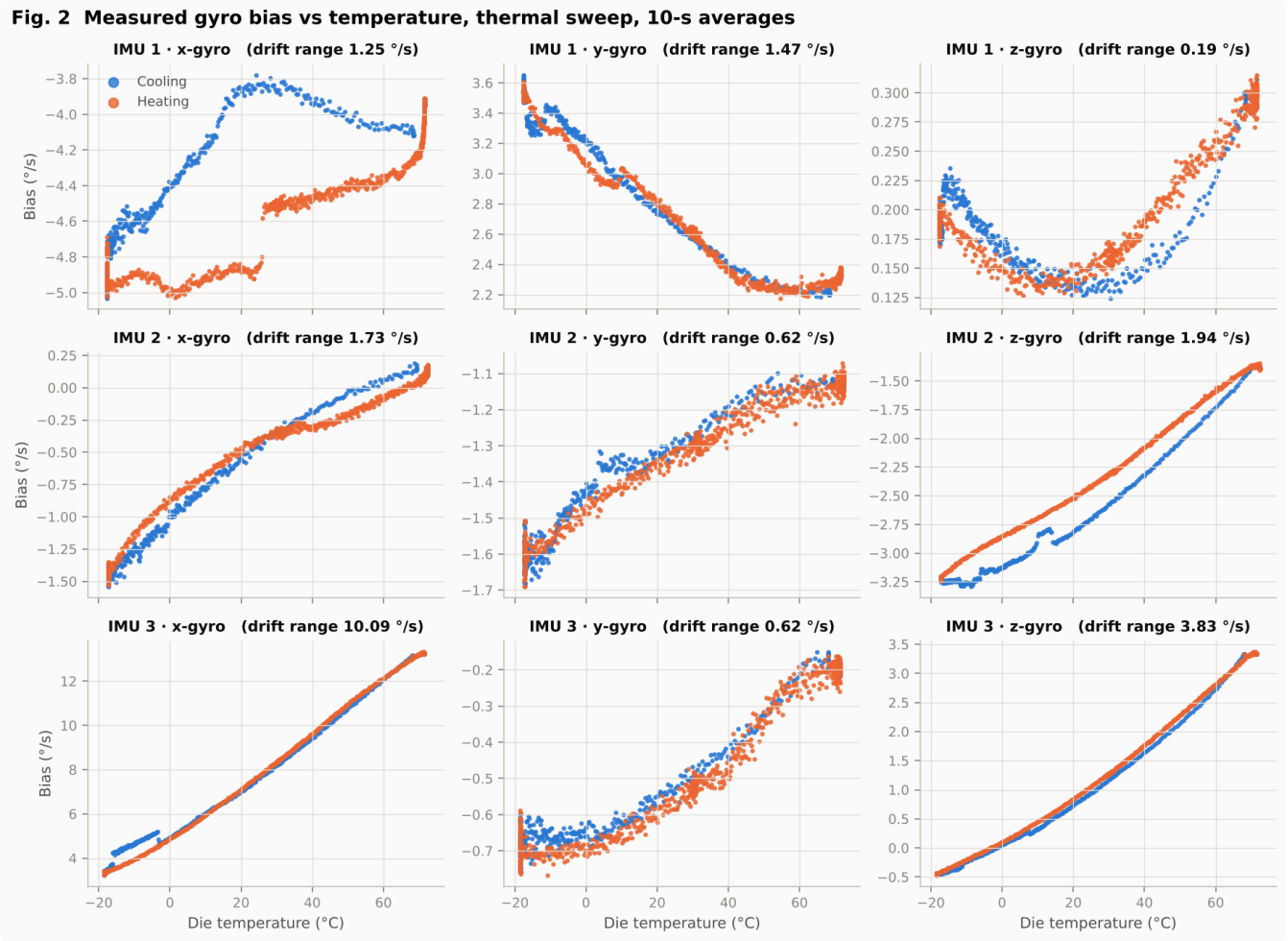
The dataset has four experiments:

Experiment	Chips	Readings per chip (about 1 per second)	What happened	What we used it for
Thermal sweep	3	10,777 (3.0 hours)	cooled from 68 to −18 °C, held, then heated to 72 °C, held	training; tests inside and outside the calibrated range
20-degree cycles	3	10,887 (3.0 hours)	warmed and cooled between 2 and 25 °C	"different day" test; data-quality study
20-degree cycles with pauses	3	17,640 (4.9 hours)	warmed and cooled between 1 and 25 °C, with pauses	"different day" test; cool ride
Humidity	5	14,298 (4.0 hours)	about 36 °C, humidity from 89 % down to 20 %	humidity study

Fig. 1 The four real chamber experiments (Cranfield MPU-6050 data)



The thermal sweep goes from colder than any Indian winter night to hotter than a bike's electronics box in the sun, so it is our main training data. Figure 2 shows how each chip's error changes with temperature. Blue dots were recorded while cooling, orange dots while heating.



What the data shows:

- **The error changes a lot, and differently for every chip.** Across the sweep it moves by 0.19 to 10.1 °/s depending on the chip and axis. On chip 3, the x-axis moves by 10.1 °/s and the z-axis by 3.83 °/s.
- **The curves are smooth and gently bent.** A curve with one bend (a quadratic) looks like a good fit; a straight line clearly misses the bend on chip 3.
- **Heating and cooling give slightly different curves.** On chip 2's z-axis the heating and cooling curves are about 0.3 °/s apart. This is called hysteresis. It means temperature alone cannot explain all the error; whether the chip is heating or cooling also matters.
- **There are sudden jumps.** Chip 1's x-axis jumps by about 0.5 °/s near 27 °C while heating. The chip maker's own engineers have reported similar unexplained jumps.

The noise limit. No model can be more accurate than the random noise in the readings. Measured while the temperature was held steady, a 10-second average still wobbles by about 0.009 °/s on the z-axis and 0.03 to 0.04 °/s on x and y. Single 1-second readings wobble about 0.027 and 0.08 °/s.

4. Cleaning the data

Three simple steps turned 189,402 raw readings into 18,966 clean 10-second averages (code: src/preprocess.py).

1. **Load and convert.** Read the four Excel files into one table and convert gyro readings from rad/s to °/s, which are easier to understand.
2. **Remove glitches.** Sometimes a logger records a nonsense value. A reading is removed if its temperature is more than 2 °C away from the readings around it, or if a gyro value is far outside the normal spread of its neighbours. This removed only 82 readings (0.04 %). The clearest example: one reading from chip 2 said 90.7 °C while the chamber was at about 18 °C.
3. **Average every 10 seconds.** Averaging 10 readings cancels much of the random noise, so the models learn the real trend instead of the noise.

The models get two inputs for each 10-second average:

Input	What it is	Why it is useful
T	the chip's own temperature, °C	the main cause of the error, and the bike can read it anytime
dT/dt	how fast the temperature is changing, °C per minute	tells the model whether the chip is heating up or cooling down, which matters because of hysteresis

We did not use the chamber's own thermometer as an input. A real bike only has the chip's built-in thermometer, so the model must work with that.

5. The models we tried

We tried seven models, from the simplest to the most flexible, and wrote each one ourselves in Python with NumPy (code: src/models.py). Each one matches a module of the course.

Model	In simple words	How it is fitted	Course module
Single-point calibration	measure the error once at 25 °C and always subtract that number	an average	1 (the baseline)

Model	In simple words	How it is fitted	Course module
Linear	error follows a straight line with temperature	normal equation	2
Polynomial	error follows a curve: T, T^2, T^3 … up to T^{10}	normal equation	3
Ridge / lasso	a polynomial with a penalty that stops it from getting too wiggly	normal equation with a penalty	3
Quadratic + heating rate	a curve in T, plus a term for heating vs cooling	normal equation	2 (several inputs)
Neural network (MLP)	2 inputs → 16 neurons → 16 neurons → 1 output	backpropagation + Adam	4
ThermoGyro (hybrid)	a quadratic curve, plus a small neural network that fixes only what the curve gets wrong	normal equation, then backpropagation	3, 4 and 5

All models try to make the **mean squared error** (the average of the squared mistakes) as small as possible, as in Module 1. Before fitting, the temperature is rescaled to have average 0 and spread 1. This "feature scaling" from Module 4 keeps the maths stable.

Straight lines and curves. These are solved exactly in one step with the normal equation from Module 2: $W = (X^TX)^{-1} X^TY$. Ridge regression adds a penalty λ to make the curve less wiggly: $W = (X^TX + \lambda I)^{-1} X^TY$. We also solved the same problem step by step with gradient descent, to compare the two methods (Section 6).

The neural network. It has 2 inputs (temperature and heating rate), two hidden layers of 16 neurons each, and one output (the predicted error). That is 337 numbers to learn. Each neuron uses the tanh activation. We wrote backpropagation by hand, using the same chain-rule formulas as Module 4, and trained it with the Adam optimiser. To make sure our backpropagation is correct, we compared its gradients with a numerical check (finite differences). They agreed to better than one part in a million.

ThermoGyro, the hybrid. It combines the strengths of both approaches:

1. **A quadratic curve does the main job.** It is simple, easy to understand and, as Section 7 shows, the most reliable model at temperatures it has not seen.
2. **A small neural network fixes the leftovers.** It is trained only on the curve's mistakes, so it learns the things a smooth curve cannot: the heating-versus-cooling difference and small jumps.

3. **A safety switch (range guard).** The neural network's fix is used only inside the temperature range the model was calibrated on. Outside that range, ThermoGyro uses the curve alone and marks the answer as "outside calibration". This follows Module 5's warning about using a model outside the conditions it was trained on.

Start-up calibration (anchoring). Each time the bike is switched on, it stands still for a few seconds and measures its current error, b_0, at the current temperature, T_0. After that the model only adds the change caused by temperature:

$$\text{error}(T) = b_0 + [\, h(T) - h(T_0) \,]$$

Section 7.3 shows that this step removes the biggest error between different days.

6. How we built and tuned them

Every setting was chosen using training data only; the test data was used just once, at the end, for the final scores. This section explains each choice, the evidence for it, and two mistakes we caught on the way.

6.1 Testing the way the bike is actually used

Readings 10 seconds apart are almost the same, so mixing them randomly into training and test sets would be cheating: the model would be tested on near-copies of what it learned. Instead we cut the data into 3-minute blocks and kept each whole block in either training or testing (blocked cross-validation). We used four tests:

Test	Learn from	Test on	Question
Inside the range	4 out of 5 groups of blocks	the 5th group	Is it accurate at temperatures it was calibrated on?
Hot test	readings at 30 °C or below	readings above 30 °C (up to 72 °C)	Calibrated in the cool, used in the heat?
Different day	the whole thermal sweep	the two other experiments (switched on at other times)	Does it still work the next day?
Different chip	two chips	the third chip	Can one chip's model be reused?

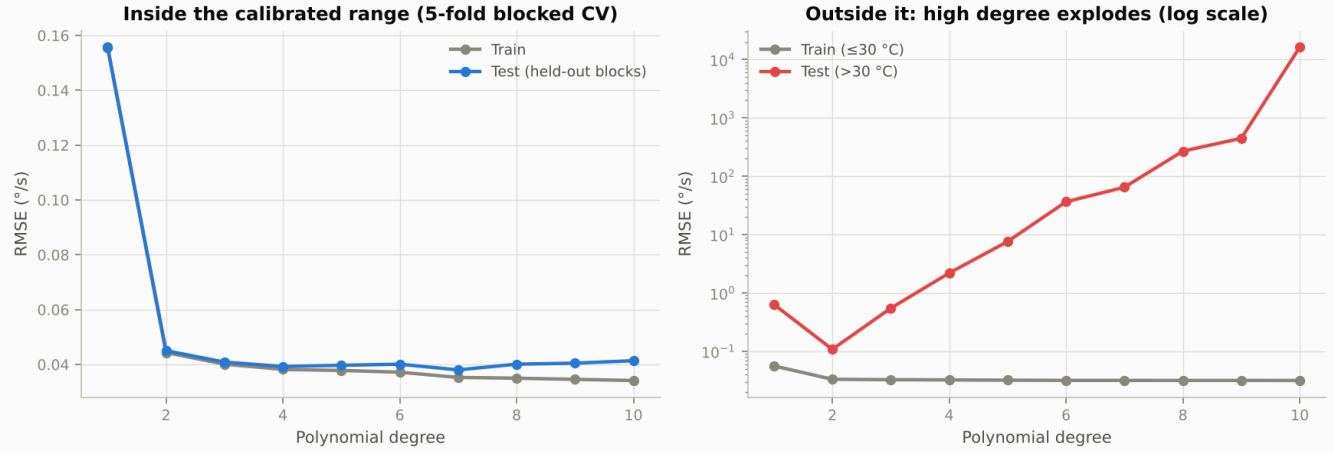
For the hot test, settings were chosen by learning on 20 °C and below and checking on 20–30 °C, which copies the real situation. The neural network stops training early (early stopping) when its error on a separate set of blocks stops improving.

6.2 How bendy should the curve be?

We chose a quadratic (a curve with one bend). In Figure 3 (left), the test error drops from 0.156 °/s with a straight line to 0.045 °/s with a quadratic, and then hardly changes (0.038 to 0.041 °/s) up to degree 10. More bends add complexity but no accuracy.

The right side of Figure 3 shows why this matters. At temperatures the model has never seen, the quadratic gives the lowest error (0.108 °/s), but degree 9 gives 447 °/s and degree 10 gives 16,306 °/s. Meanwhile the training error keeps getting smaller. This is the bias–variance trade-off from Module 3, seen on real data: a curve that fits the training data perfectly can be terrible on new data.

Fig. 3 Bias-variance in action, IMU 3 z-gyro



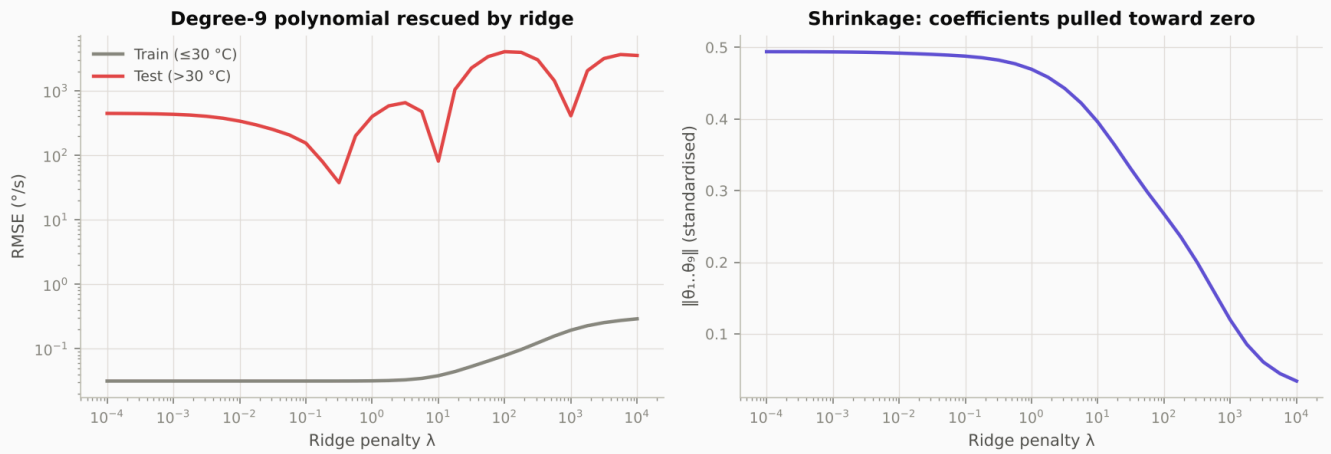
High-degree curves are also numerically unstable: the condition number of X^TX grows from 15 at degree 2 to 24 billion at degree 10.

We also learned a lesson from letting the computer choose the degree automatically. When it checked only the 20–30 °C range, it picked a straight line for four of the nine chip-axes, and degree 5 once. For chip 2 x that gave 1.785 °/s of error in the heat, against 0.109 °/s for the quadratic. A 10 °C check cannot tell you how a curve will behave 40 °C further away. Knowing the physics (the curve should be smooth and gently bent) has to help when a model will be used beyond its data.

6.3 Regularisation

Ridge regularisation helped the wild degree-9 curve, but could not save it. It cut the hottest error from 445 °/s to 37.6 °/s, about 12 times better, by shrinking the curve's coefficients towards zero (Figure 4). Even so, it was still 350 times worse than the simple quadratic. Regularisation reduces wiggles, but it cannot guess the right shape outside the data.

Fig. 4 Regularisation, IMU 3 z-gyro



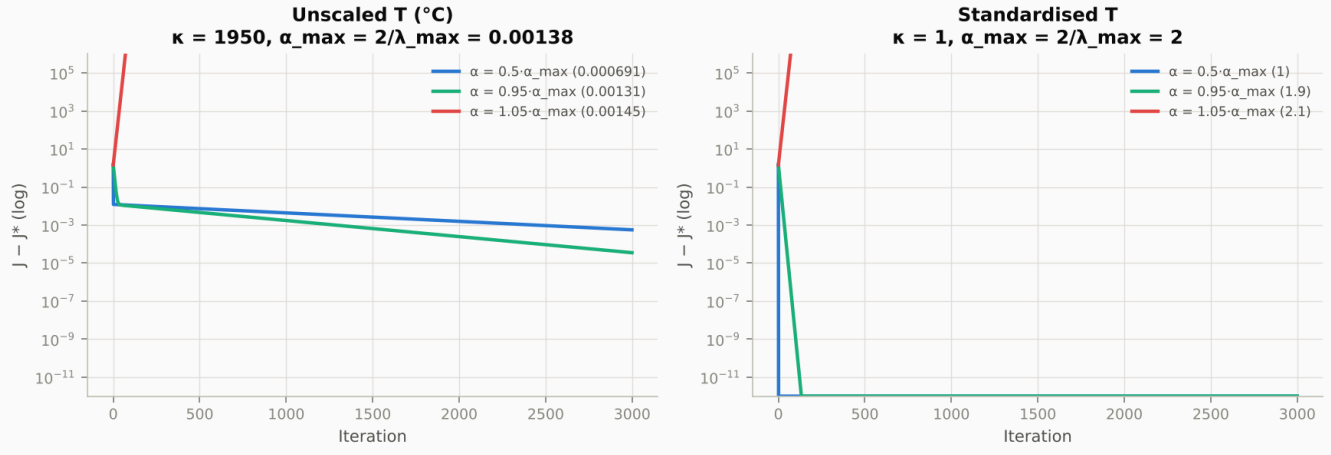
Lasso (the other penalty) kept only three of the nine terms: T, T^5 and T^9. The T^5 and T^9 terms have no physical meaning, which is another reason to choose the simple physics-shaped curve ourselves.

6.4 Gradient descent versus the normal equation

Gradient descent finds the same answer as the normal equation, but only when the input is scaled (Figure 5).

- **Without scaling (temperature in °C):** the condition number is 1,950 and the learning rate must stay below 0.00138. Even close to that limit, after 3,000 steps the answer is still not quite right (intercept 0.172 instead of 0.182). Just 5 % above the limit, the cost explodes to 1.2×10^{12}.
- **With scaling:** the condition number is 1 and the learning rate can go up to 2. At half that, gradient descent gets the exact answer in one step; close to the limit it takes 88 steps.

Fig. 5 Gradient descent on the linear model, IMU 3 z-gyro (real data)



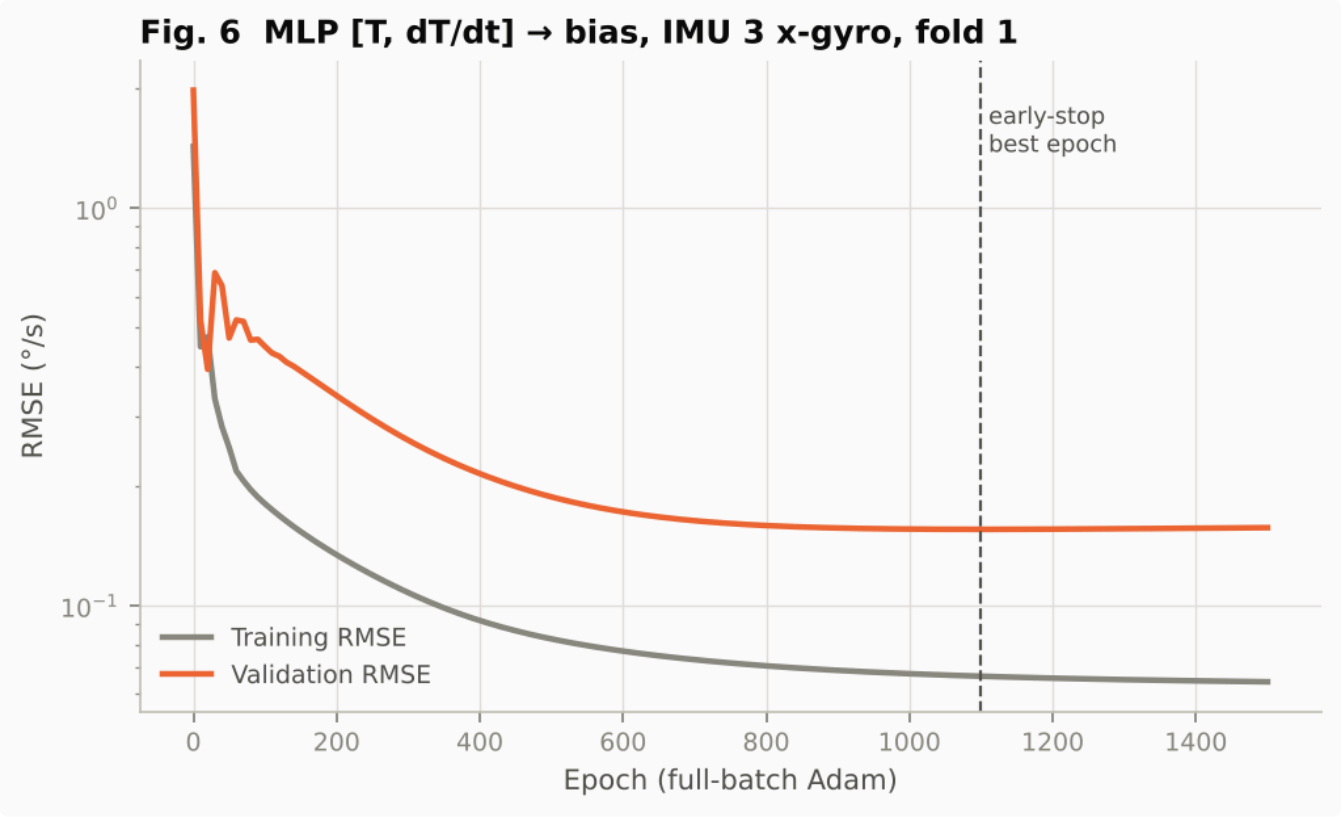
This matches the course rule exactly: gradient descent blows up when the learning rate goes above 2 ÷ (largest eigenvalue of X^TX/N).

6.5 Tuning the neural network

We tried several network sizes and activations on chip 3 x, the axis with the clearest heating/cooling difference. Two layers of 16 tanh neurons worked best.

Activation	Hidden layers	Numbers to learn	Training error (°/s)	Test error (°/s)	Dead neurons
tanh	4	17	0.173	0.178	0
tanh	8, 8	105	0.117	0.146	0
tanh	**16, 16**	**337**	**0.090**	**0.102**	**0**
tanh	32, 32	1,185	0.102	0.122	0
ReLU	16, 16	337	0.138	0.141	1
ReLU	32, 32	1,185	0.110	0.134	6.4
ReLU, started badly (biases at −10)	16, 16	337	3.623	3.712	32 (all)

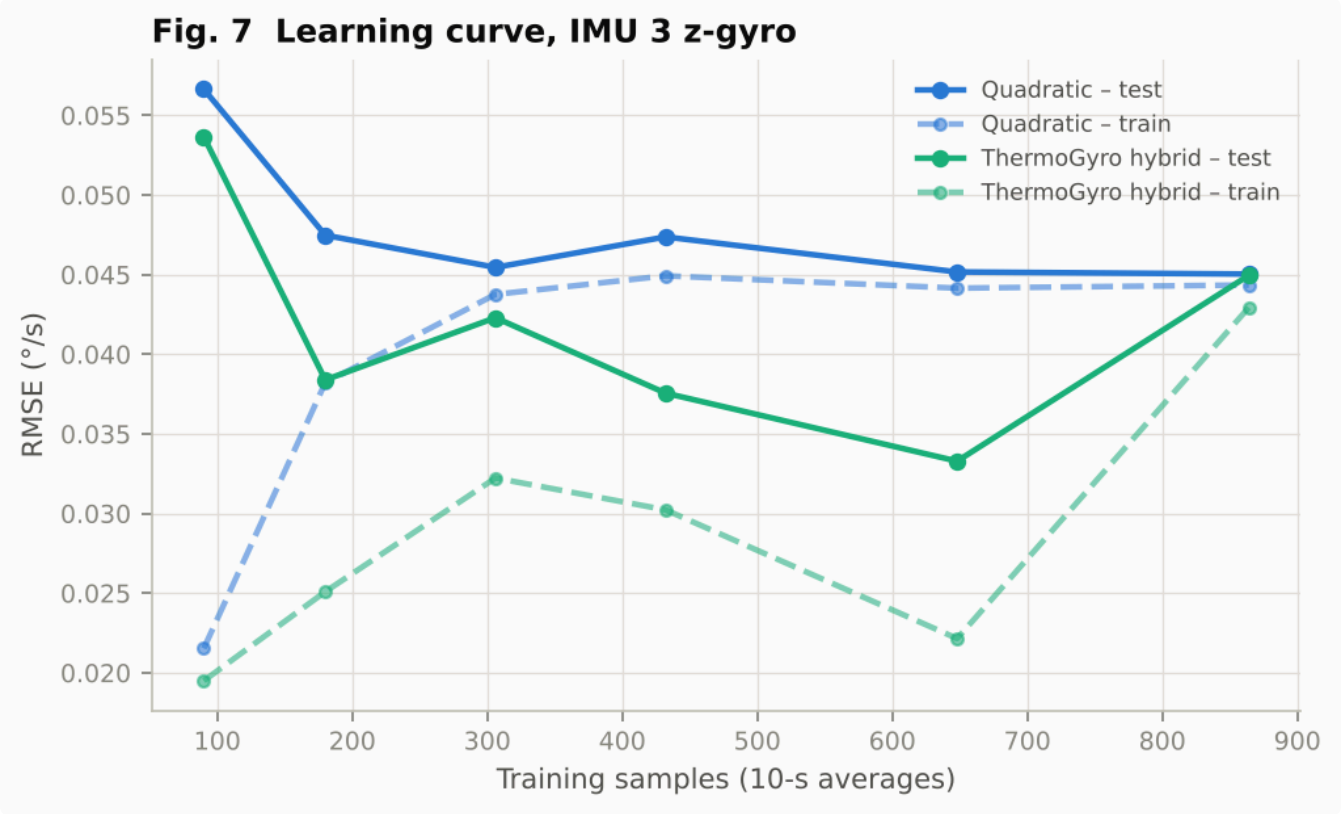
The last row shows the "dying ReLU" problem from Module 4 on purpose. When every hidden neuron starts with a bias of −10, all 32 of them output zero for every input, no learning signal reaches them, and the network can only predict a constant (error 3.71 °/s). Even with a good start, the larger ReLU network lost about 6 of its 64 neurons. tanh never fully "dies" and fit this smooth problem better.



Training stopped after about 500 to 1,950 rounds (epochs), depending on the network: 1,367 on average for the chosen one.

6.6 How much data does it need?

The hybrid needs about 30 minutes of calibration data (180 ten-second readings) before it beats the simple curve (Figure 7). With only 90 readings its error is 0.105 °/s against 0.057 °/s for the quadratic: with too little data the neural network adds noise, not accuracy. From 180 readings onwards it is about as good or better, and with 648 readings it reaches 0.032 against 0.045 °/s.



6.7 Two mistakes we caught

We found both by checking results that looked wrong, and we kept them in the report.

1. **The curve was missing some data.** In the first version, the hybrid's quadratic was fitted without the blocks set aside for the network's early stopping. For chip 1 x this changed the curve's bend and the hot-test error jumped from 0.226 to 0.839 °/s. The fix: the curve now uses all training data, and the set-aside blocks are used only to stop the network. This also showed how fragile predictions outside the data are: leaving out a different one-seventh of the data each time gave hot-test errors from 0.218 to 0.841 °/s for chip 1 x.

2. **The safety switch had a gap.** At first it let the neural fix work up to 2 °C outside the calibrated range. In the hot ride, which starts at 31 °C, this made the hybrid look better than it really was. We set the gap to 0 °C: outside the calibrated range, ThermoGyro is exactly the quadratic.

6.8 Final settings

Setting	Value	Chosen by
Averaging	10 seconds	noise vs. how fast temperature changes
Main curve	quadratic	elbow of the test-error curve (Figure 3)
Neural network	[T, dT/dt] → 16 tanh → 16 tanh → 1	test error (Section 6.5)
Training	Adam, learning rate 0.003, small weight penalty, up to 3,000 epochs	stable validation error
Early stopping	stop after 400 epochs with no improvement	validation
Safety switch	neural fix off outside the calibrated temperatures	Section 6.7
Start-up calibration	60 s still at switch-on in tests (3 s in the bike code)	Section 7.3

7. Results

No single model wins in every situation. The neural models are best at temperatures they were calibrated on, the simple quadratic is best outside them, and the start-up calibration matters most from one day to the next. That is why ThermoGyro combines all three.

All numbers below are **RMSE in °/s**: the typical size of the leftover error on test data the model never saw. Smaller is better. The best value in each row is in bold. Full tables are in the results/ folder.

7.1 At temperatures it was calibrated on

When the training data includes both heating and cooling, the models that also look at the heating rate capture the heating-vs-cooling difference and beat the simple curve.

Chip, axis	Single-point	Straight line	Quadratic	Quadratic + heating rate	Neural network	ThermoGyro
Chip 1, x	0.392	0.232	0.231	0.105	**0.072**	0.076
Chip 1, y	0.538	0.129	0.071	0.071	0.082	**0.037**
Chip 1, z	0.074	0.040	0.015	0.013	0.010	**0.008**
Chip 2, x	0.642	0.119	0.060	0.046	0.032	**0.029**
Chip 2, y	0.194	0.041	0.030	0.028	0.026	**0.025**
Chip 2, z	0.699	0.107	0.090	0.038	**0.023**	0.025
Chip 3, x	3.651	0.277	0.194	0.183	**0.102**	0.111
Chip 3, y	0.195	0.056	0.034	0.032	**0.027**	0.031
Chip 3, z	1.394	0.156	0.045	0.034	**0.033**	0.045

What this shows:

- ThermoGyro beats the quadratic on all 9 chip-axes (typically 1.75 times smaller error) and beats single-point calibration by about 14 times.
- The neural network alone is just as good here: it beats the quadratic on 8 of 9, and the two neural models share the wins (5 to 4).
- On chip 1 z, ThermoGyro's error (0.008 °/s) is as small as the noise in the data (0.009 °/s). There is nothing more to learn there.
- Just adding the heating rate to the simple curve halves the error on chip 2 z (0.090 → 0.038). This proves the improvement comes from knowing whether the chip is heating or cooling.

7.2 Calibrated in the cool, used in the heat

When the models learn only from 30 °C and below and are tested from 30 to 72 °C, the ranking flips: simple models survive and flexible models fail.

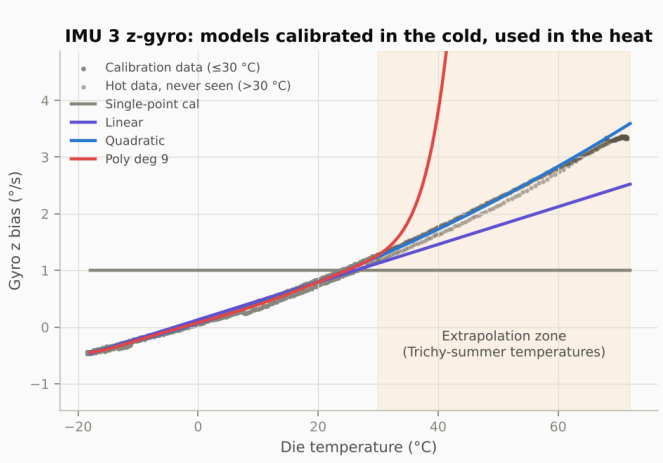
Chip, axis	Single-point	Straight line	Quadratic	Degree-9 curve	Neural network	ThermoGyro
Chip 1, x	0.259	0.225	0.226	16,792	**0.123**	0.226
Chip 1, y	0.364	0.331	**0.135**	1,299	0.237	**0.135**
Chip 1, z	0.106	0.156	**0.057**	40.8	0.095	**0.057**
Chip 2, x	0.377	0.507	**0.109**	3,399	0.150	**0.109**
Chip 2, y	0.150	**0.119**	0.135	3,019	0.137	0.135
Chip 2, z	0.852	0.319	**0.144**	6,823	0.637	**0.144**
Chip 3, x	4.129	1.018	**0.196**	8,669	2.960	**0.196**

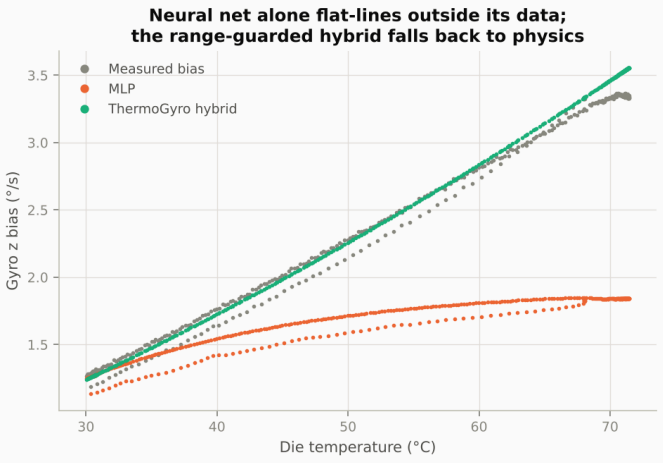
Chip, axis	Single-point	Straight line	Quadratic	Degree-9 curve	Neural network	ThermoGyro
Chip 3, y	0.272	0.199	**0.104**	515	0.178	**0.104**
Chip 3, z	1.698	0.635	**0.108**	447	1.034	**0.108**

What this shows:

- The quadratic beats single-point calibration on all 9 chip-axes (about 2.7 times better).
- The neural network beats the quadratic only once (chip 1 x, where the quadratic itself is shaky, Section 6.7). On chip 3 x it is 15 times worse.
- Figure 8 shows why. A neural network has no idea what happens beyond its data, so its prediction goes flat, while the real error keeps rising. Because of its safety switch, ThermoGyro uses the quadratic here, so it gives exactly the quadratic's result.

Fig. 8 Cool-weather calibration, hot-weather riding (real data)





7.3 A different day

From one day to the next, the biggest change is not the shape of the curve but a fixed jump every time the sensor is switched on. At the same temperature, the error in the other two experiments differed from the thermal sweep by up to 2.7 °/s (chip 3 x), and by more than 0.25 °/s in 9 of the 18 cases. This is why ThermoGyro measures the error for a few seconds at every start-up.

Trained on the thermal sweep and tested on the "cycles with pauses" experiment (with the first 60 seconds used for start-up calibration):

Chip, axis	Start-up calibration only	Straight line	Quadratic	Neural network	ThermoGyro
Chip 1, x	0.414	0.392	**0.389**	0.551	0.548
Chip 1, y	**0.109**	0.185	0.206	0.263	0.183
Chip 1, z	0.046	**0.045**	0.049	0.056	0.052
Chip 2, x	0.223	0.104	0.083	0.077	**0.065**

Chip, axis	Start-up calibration only	Straight line	Quadratic	Neural network	ThermoGyro
Chip 2, y	0.041	**0.022**	0.028	0.028	0.025
Chip 2, z	0.193	**0.053**	0.058	0.082	0.060
Chip 3, x	0.883	0.115	**0.114**	0.210	0.150
Chip 3, y	**0.026**	0.047	0.038	0.040	0.039
Chip 3, z	0.383	**0.115**	0.135	0.149	0.128

What this shows:

- Where the error changes a lot with temperature, the learned curve helps a lot: chip 3 x improves from 0.883 to 0.114 °/s, and chip 2 z from 0.193 to about 0.055 °/s.
- Where the error barely changes (chip 1 y, chip 3 y), start-up calibration alone is best, because the curve's shape itself changed a little between days.
- ThermoGyro's extra accuracy from Section 7.1 does not reliably carry over to another day: it beats the quadratic on 4 of 9 here and 3 of 9 on the other experiment.
- The start-up calibration is what matters most. Without it, the quadratic's typical error across both days is 0.230 °/s; with it, 0.107 °/s.

7.4 A different chip

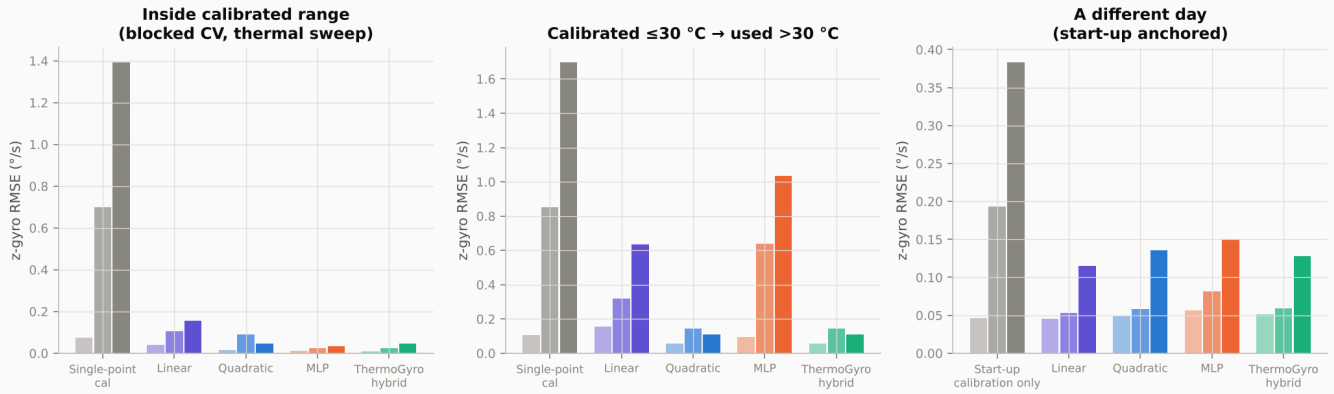
One chip's model does not work on another chip. A model made from two chips gave 1.3 to 10.5 °/s of error when used on the third. Measuring the new chip once at 25 °C and borrowing only the curve's shape helped on two of three chips for the z-axis, but was still much worse than the chip's own calibration. Every MPU-6050 needs its own calibration, which is why the web app lets you calibrate your own sensor.

Test chip, z-axis	Other chips' model as it is	New chip measured once at 25 °C	Measured once + borrowed shape	Chip's own quadratic
Chip 1	1.363	0.074	1.005	0.015
Chip 2	3.125	0.698	0.098	0.090
Chip 3	2.399	1.394	1.032	0.044

The last column is fitted and scored on the same data, so it shows the best possible case, not a fair test.

7.5 Overview

Fig. 9 Model comparison, z-gyro of all three chips (bars left→right: IMU 1, 2, 3)



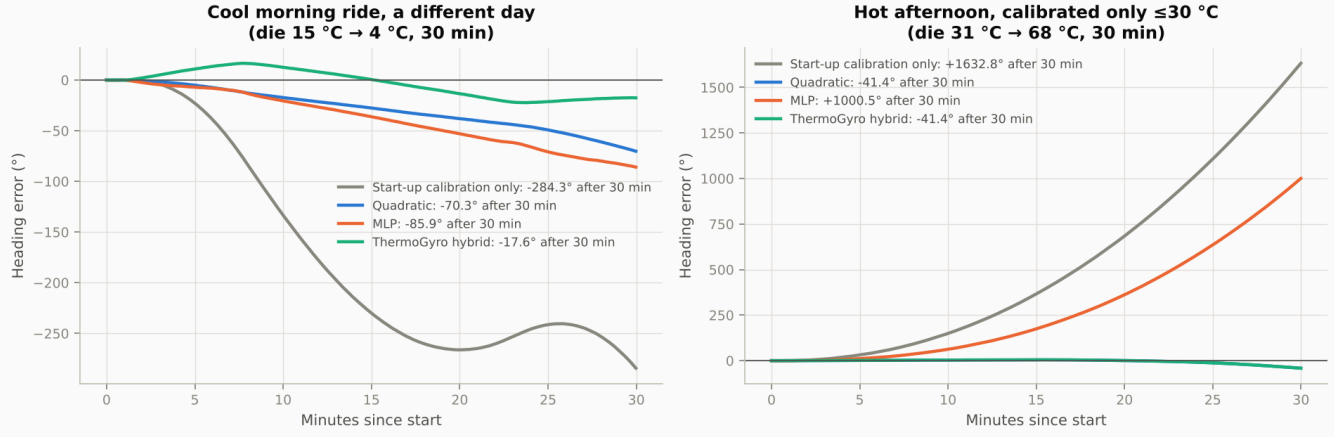
7.6 What the rider would see

The result that matters most is the direction error of a bike that is standing still. We got it by adding up the real 1-second gyro readings of chip 3's z-axis over 30 minutes.

30-minute ride	No correction	Start-up calibration only	Quadratic	Neural network	ThermoGyro
Cool morning, another day (15 → 4 → 8 °C)	1,310°	−284°	−70°	−86°	**−18°**
Hot afternoon, calibrated at 30 °C or below (31 → 68 °C)	4,025°	1,633°	**−41°**	1,001°	**−41°**

- **Hot afternoon:** with start-up calibration only, the still bike believes it has spun around four and a half times. The neural network alone is not much better. ThermoGyro keeps the error to 41°, about 40 times smaller.
- **Cool morning:** this is inside the calibrated range, so ThermoGyro's heating/cooling fix works, giving the best result: 18° against 70° for the quadratic.
- The error is still not zero, so a real bike should also use GPS or a compass to correct its direction. ThermoGyro makes the gyro much better to combine with them.

Fig. 13 What the rider sees: heading error from integrating the real, still gyro (IMU 3 z)



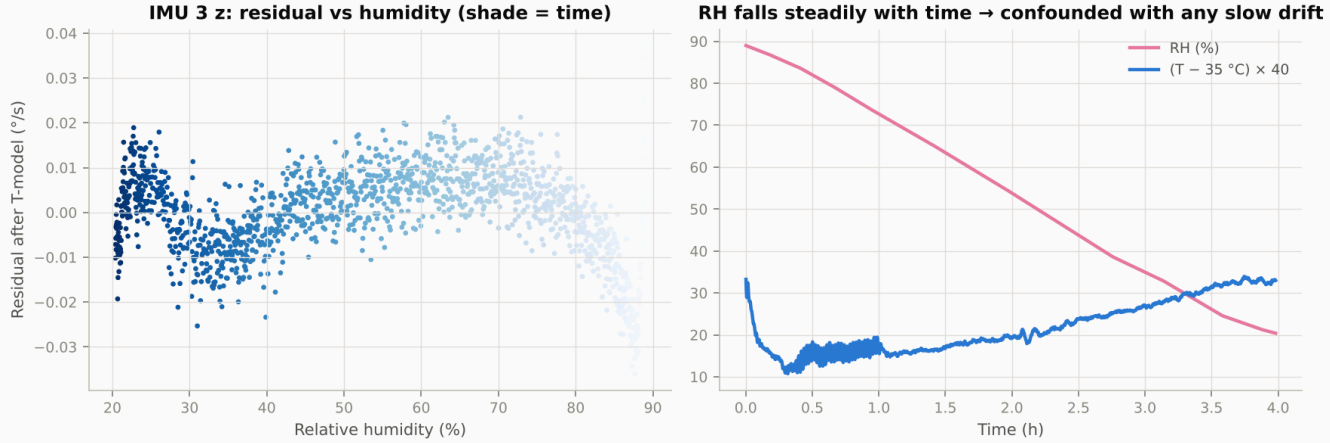
8. What did not work, and the limits

The data gives clear answers about temperature, but not about humidity, and a few bad readings can badly bend a fit. This section checks the project against Module 5's list of ways data-driven models can go wrong.

8.1 Humidity: a pattern we cannot trust

At first, humidity seemed to help. Adding it as an input lowered chip 1 x's error from 0.117 to 0.052 °/s. But in that experiment, humidity dropped steadily as time went on (correlation −0.999), and it also moved with the small temperature changes (correlation −0.84 to −0.89). Humidity and time are almost the same signal, so the model cannot tell them apart: adding time as well leaves the error at 0.052 °/s. Any slow drift of the sensor over time would look like a humidity effect.

Fig. 10 Does humidity matter? (5 chips at ~36 °C, RH 89% → 20%)



This is the "spurious correlation" trap from Module 5. To really test humidity, it would have to change on its own while time and temperature stay the same. So this project makes **no claim** that humidity affects the MPU-6050, and ThermoGyro does not use humidity.

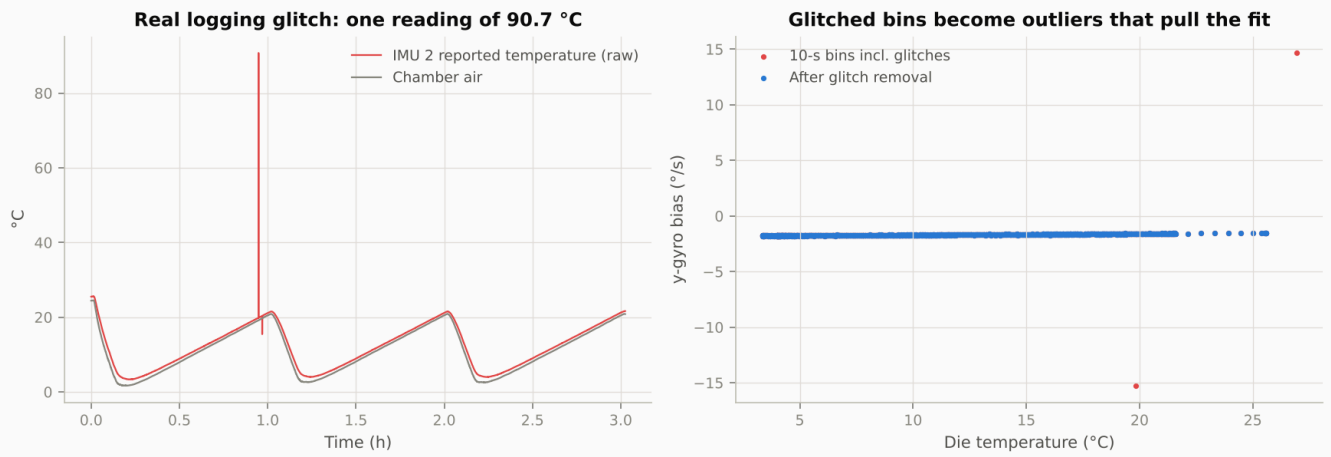
8.2 Three bad readings, one badly bent curve

The 20-degree-cycles experiment contains a few logging glitches, including chip 2's 90.7 °C reading. We fitted chip 2's y-axis quadratic with and without them:

Training data	How much the curve bends (θ_2)	Error on another day (°/s)
With the glitches	0.092	0.189
Glitches removed (3 of 10,887 readings)	0.005	0.054

Just 3 bad readings out of 10,887 made the curve bend 17 times more and made the next-day error 3.5 times worse. The x and z axes were hardly affected. This is why the data is cleaned before any fitting, as Module 5 recommends.

Fig. 11 Data quality: a few bad samples out of 10,887



8.3 Other limits

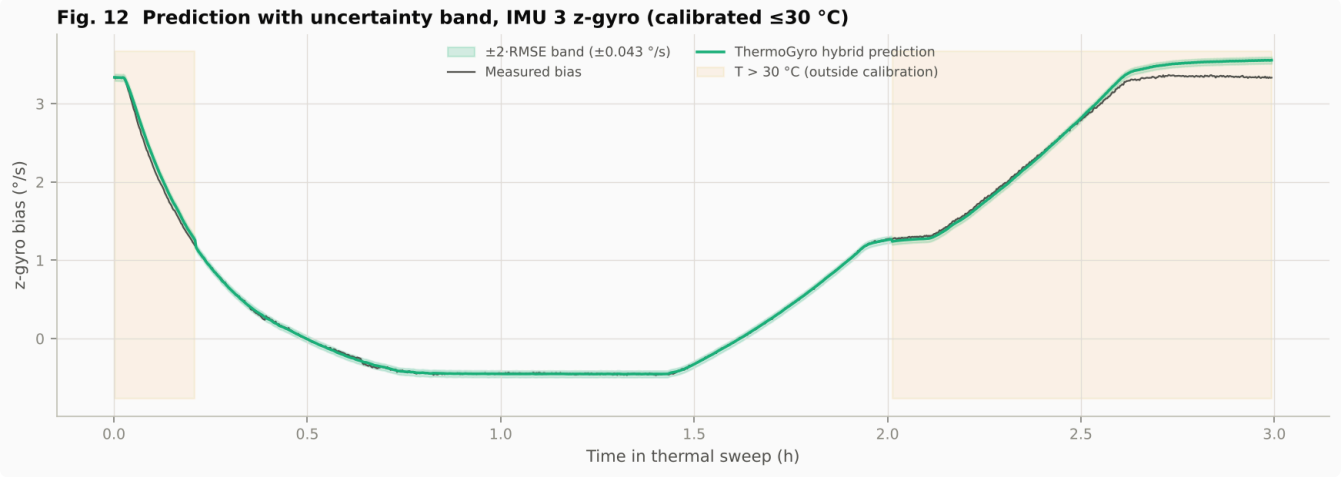
- **Predicting outside the data is fragile, even for the best model.** Leaving out a different one-seventh of the calibration data changed the quadratic's hot-test error for chip 1 x from 0.22 to 0.84 °/s. Calibrating over the whole temperature range you will use is much safer than relying on extrapolation.
- **Only three chips, from one lab.** The results describe these three MPU-6050s. Other chips will have other curves. What carries over is the method, not the exact numbers.
- **Only still sensors.** The chips never rotated, so this study covers the zero error, not how the sensitivity (scale factor) changes with temperature while the bike is actually turning.
- **Jumps and heating/cooling effects are only partly learned.** The hybrid learns them inside the calibrated range, but the jump on chip 1 x near 27 °C and the small change in curve shape between days show behaviour no model of temperature and heating rate can fully predict.
- **The chip's thermometer is not exactly the gyro's temperature.** The model can only use what the chip reports. Any delay between the two is part of what the neural network tries to learn.

9. How sure are the predictions?

The error band from Module 5 (prediction ± 2 × RMSE) is trustworthy inside the calibrated range but far too confident outside it. We set each band's width from validation data only, then counted how often the real test readings actually fell inside it.

Model	Typical band width	Readings inside the band: calibrated range	30–40 °C	40–72 °C
Quadratic	±0.125 °/s	93 % (85–98 %)	100 % (88–100 %)	52 % (9–100 %)
ThermoGyro	±0.064 °/s	96 % (88–98 %)	75 % (0–96 %)	34 % (8–44 %)

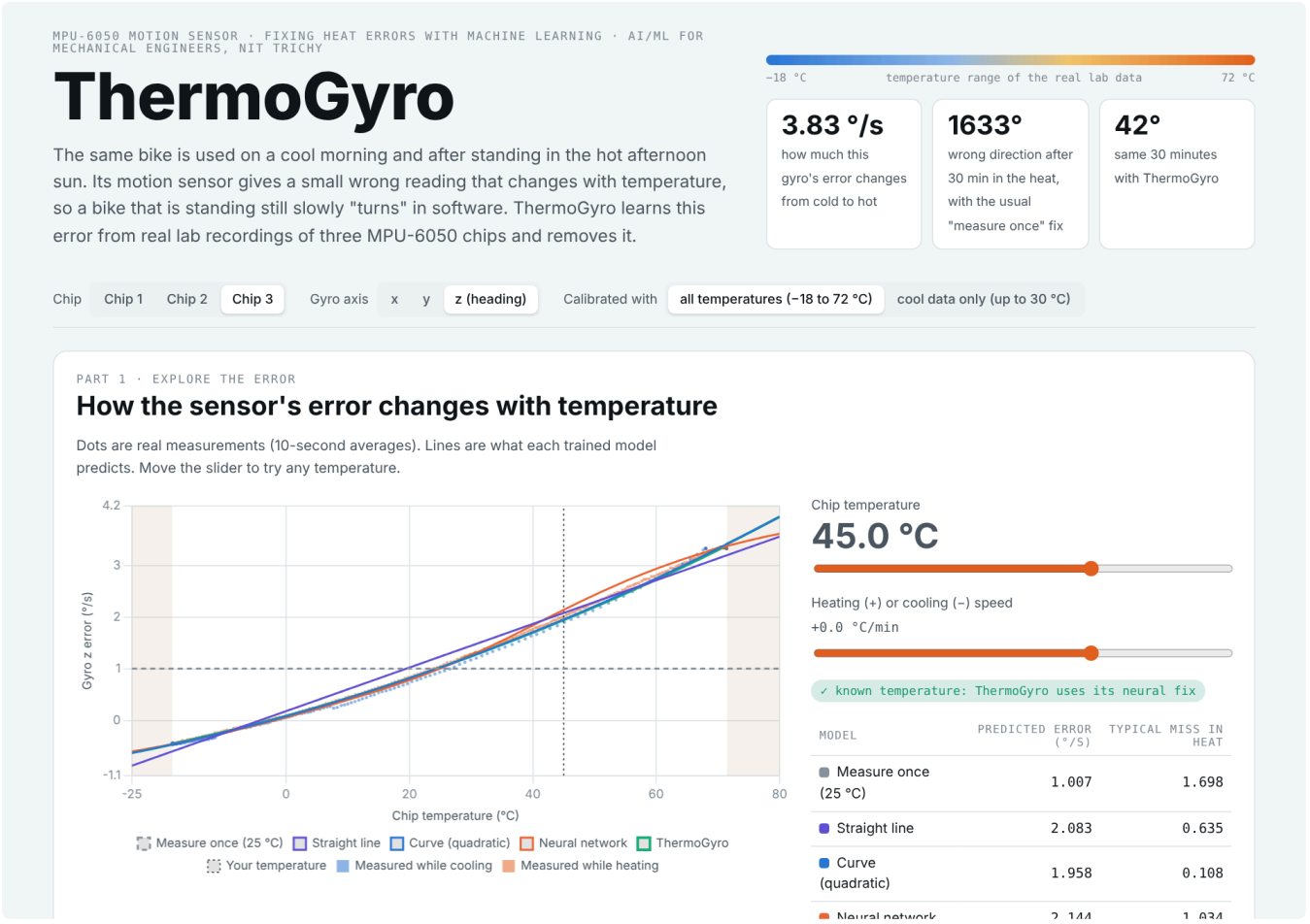
Inside the calibrated range, ThermoGyro's band is half as wide as the quadratic's and still contains 96 % of the real readings, close to the 95 % you would expect. Outside the range, the same band contains only about a third of the hot readings. The band was learned where the model is accurate, so it says nothing about how wrong the model can be beyond its data, exactly as Module 5 warns.



So the practical rule is: **when the temperature goes outside the calibrated range, mark the prediction as uncertain.** The web app shows a warning label when this happens, and the generated bike code says to recalibrate over a wider range.

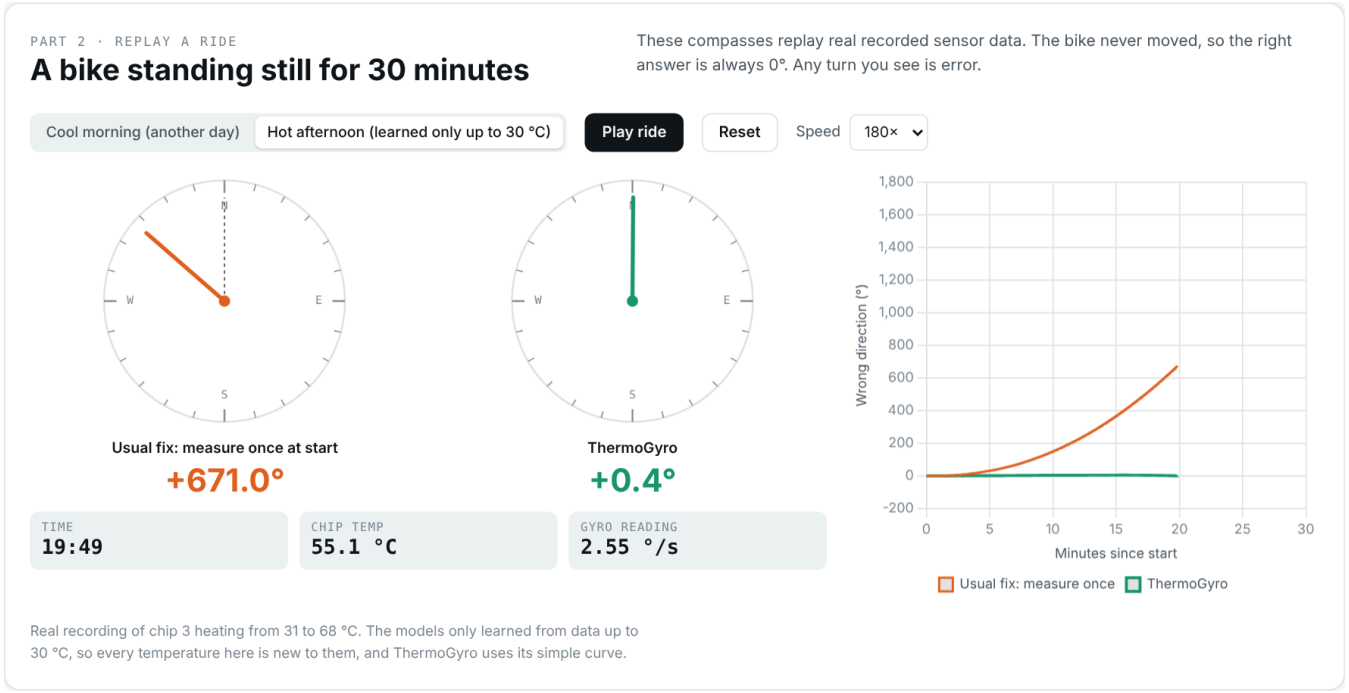
10. The app and the code for the bike

The trained models are usable in two ways: a web app that runs them on the real recordings, and code for the ESP32 board on the bike. The app is a single web page in the repository's docs/ folder. It runs in any browser with no server, because the trained model numbers run directly in JavaScript, and it can be published for free with GitHub Pages.



The app has four parts:

1. **Explore the error.** Choose a chip, an axis and how it was calibrated, then move the temperature and heating-rate sliders. Every model's prediction updates live, and a label shows whether ThermoGyro is inside its calibrated range or has switched to the quadratic.
2. **Replay a ride.** Two compasses replay the real 30-minute recordings: "start-up calibration only" against ThermoGyro, for the cool ride and the hot ride (Figure 15).
3. **Calibrate your own sensor.** Upload a log of your own still MPU-6050 (time, temperature, gx, gy, gz). The page averages it, fits a quadratic for each axis with the normal equation, shows how good the fit is, and writes ready-to-paste code for the ESP32.
4. **See the evidence.** The results tables from Section 7 for the chosen chip and axis.



We checked that the app gives the same answers as the Python code:

Check	Result
App predictions vs Python, 828 predictions (9 chip-axes × 4 model types × 23 inputs)	the biggest difference is 0.0000000000000018 °/s, i.e. identical
Ride replay vs Python	+1,632.8° vs +1,632.8° and −41.9° vs −41.4° (hot); −17.3° vs −17.6° (cool); the tiny gap comes from a one-reading difference in how temperature is smoothed
Calibrating a real 17,640-reading log in the app vs Python	same R^2 and RMSE to 4 decimal places on all three axes
Phone screen (400 px wide), light and dark mode	fits the screen, nothing cut off

Code for the bike. The firmware/ folder has two Arduino programs for the ESP32:

- **Logger:** prints the sensor's temperature and gyro readings in exactly the format the app accepts, so you can record your own chip.
- **Compensation:** at switch-on it measures the error for 3 seconds while the bike is still, then uses the fitted curve to correct the gyro and track direction.

Both compile without errors or warnings, but they have **not yet been run on a real board**, because no hardware was available before the deadline. Running the logger on my own bicycle's MPU-6050 is the first next step.

11. Using AI responsibly

Before calling ThermoGyro finished, we went through Module 5's ten-question checklist (Section 5.8.2).

#	Question	ThermoGyro's answer
1	Where did the data come from?	A public Cranfield University dataset, cited, free to use (CC BY 4.0). It represents these three chips standing still, not every MPU-6050 or a moving bike.
2	Do the results make physical sense?	Yes: the fitted curves are smooth and gently bent, as the physics suggests. The lasso's T^5 and T^9 terms were rejected because they have no physical meaning (Section 6.3).
3	Is it used only where it was tested?	The safety switch turns off the neural part outside the calibrated temperatures, and the app labels every such prediction.
4	Was it tested on unseen data?	Yes, four separate tests on data the model never saw. Training error was never used to pick a model.
5	Is the uncertainty shown?	Yes, error bands with measured coverage: 96 % inside the range, about 34 % above 40 °C (Section 9).
6	Is it simple enough for the risk?	The bike runs a 3-number curve per axis; the neural fix is optional and limited. A wrong direction on a bike is a nuisance, not a danger, but the same care would matter a lot on a drone.
7	Does a person stay in control?	The calibrator shows the fit quality and the curve before writing any code, so the user decides if the fit is good enough.
8	Can someone else repeat it?	One command (run_all.sh) rebuilds every table and figure from the raw files. Two full runs gave identical numbers.
9	Privacy and ownership?	No personal data; the dataset's licence is followed with credit; our code is open (MIT licence).

#	Question	ThermoGyro's answer
10	Who is responsible?	The report states clearly what was not tested: no real-board run, only three chips, no humidity conclusion.

The most important responsible decision was to **not** use the neural network on its own. It looked like the best model at calibrated temperatures, but the hot test showed it would badly mislead a bike in summer. So it is only ever used together with the physics curve and the safety switch.

12. What we learned and what comes next

A cheap gyro's temperature error can be learned and removed using real data, but it works reliably in both cool and hot weather only when physics, machine learning and a start-up measurement are combined. On real recordings of a still MPU-6050, ThermoGyro reduced the hot-afternoon direction error from 1,633° to 41°, and the cool-morning error on another day from 284° to 18°.

Six lessons, each one measured on real data:

1. **Fitting the training data better does not mean a better model.** Degree-9 curves fit the calibration data almost perfectly, then gave errors of up to 16,792 °/s in the heat.
2. **Neural networks are good inside their data and bad outside it.** The network was among the best at calibrated temperatures and up to 15 times worse than a simple curve beyond them.
3. **Physics knowledge is part of the model.** The simple curve and the safety switch are what make the hybrid safe to use.
4. **The biggest day-to-day error is a jump at switch-on.** Measuring the error at every start-up halved the error between days.
5. **Clean data matters more than a bigger model.** Three bad readings out of 10,887 made one curve bend 17 times more.
6. **A pattern is not a cause.** Humidity looked useful but could not be told apart from time.

Next steps, most valuable first:

- **Calibrate my own bicycle's sensor.** Run the logger on the real board, warm it up and cool it down, fit it in the app, and ride with the correction code.
- **Let parked bikes calibrate themselves.** A docked bike stands still for hours while the day heats up and cools down, so it can record its own error curve and keep its model up to date.

- **Combine with GPS.** Mix the corrected gyro with the GPS direction (for example with a Kalman filter) to remove the remaining tens of degrees while riding.
- **Test more chips, and test while rotating.** More sensors would show what carries over between chips, and a turntable test would cover the sensitivity too.

13. How to re-run everything, and references

Every number in this report comes from code in the GitHub repository, and one command rebuilds all of it:

```
pip install -r requirements.txt
./run_all.sh                        # cleans the data, runs the tests and
every study (~3 minutes)
cd docs && python -m http.server    # then open the app at
http://localhost:8000
```

Folder	What is inside
data/raw/	the four original Cranfield Excel files
src/	loading, cleaning, the models (written from scratch) and all experiments
results/	every table in this report, as CSV or JSON
figures/	every figure in this report
tests/	checks: backpropagation gradients, normal equation, gradient descent, app vs Python
docs/, web/	the web app and its source
firmware/	ESP32 logger and correction programs

Where each part of the course is used:

Module	Topic	Where in ThermoGyro
1	Data-driven vs physics-based models; hypothesis; mean squared error	Sections 2 and 5; the quadratic as the physics part
2	Normal equation, several inputs, R^2, RMSE	straight line, quadratic, quadratic + heating rate; the app's calibrator

Module	Topic	Where in ThermoGyro
3	Curves (basis functions), bias–variance, learning curves, ridge and lasso, gradient descent, learning rate	Sections 6.2 to 6.4 and 6.6; Figures 3 to 5 and 7
4	Neurons, activations, neural networks, backpropagation, dying ReLU, Adam, feature scaling	Sections 5 and 6.5; gradient check; Figure 6
5	Using models outside their data, data quality, spurious correlation, error bands, responsible-AI checklist	Sections 7.2, 8, 9 and 11; the safety switch

References

1. D. Maton, J. Economou, D. Galvão Wall, I. Khan, R. Cooper. Low-cost IMU Sensor Temperature Humidity Zero Bias Data. Cranfield University, 2024. CC BY 4.0.
2. InvenSense. MPU-6000 and MPU-6050 Product Specification, revision 3.4.
3. InvenSense developer forum. MPU-6050 thermal chamber results: unexplained bias behaviour and hysteresis.
4. Course notes: Introduction to AI & Machine Learning for Applications in Mechanical Engineering, Modules 1–5, NIT Tiruchirappalli.
5. Chart.js 4.4.1 (MIT licence), used for the app's charts.

The code, app and this report were developed with help from Claude (Anthropic). All results come from running that code on the cited data.